

Содержание

1	Исследование процессов обнаружения, сопряжения и установления соединения в сетях Bluetooth	2
1.1	Цель работы	2
1.2	Задачи	2
1.3	Теоретический материал	3
1.4	Порядок выполнения лабораторной работы	5
1.5	Содержание отчета	7
2	Исследование архитектуры пикосети Bluetooth и организация обмена данными через RFCOMM	9
2.1	Цель работы	9
2.2	Задачи	9
2.3	Теоретический материал	9
2.4	Порядок выполнения лабораторной работы	12
2.5	Содержание отчета	15
3	Анализ трафика Bluetooth с использованием Wireshark	17
3.1	Цель работы	17
3.2	Задачи	17
3.3	Теоретический материал	17
3.4	Порядок выполнения лабораторной работы	19
3.5	Содержание отчета	22
4	Организация удаленного выполнения команд через Bluetooth-мост на базе ESP32	24
4.1	Цель работы	24
4.2	Задачи	24
4.3	Теоретический материал	24
4.4	Порядок выполнения лабораторной работы	26
4.5	Содержание отчета	28
5	Исследование архитектуры Bluetooth Low Energy (BLE) и реализация кастомных профилей GATT	30

5.1	Цель работы	30
5.2	Задачи	30
5.3	Теоретический материал	30
5.4	Порядок выполнения лабораторной работы	34
5.5	Содержание отчета	36
6	Сравнение скоростных характеристик протоколов Bluetooth (BR, EDR, BLE)	38
6.1	Цель работы	38
6.2	Задачи	38
6.3	Теоретический материал	38
6.4	Порядок выполнения лабораторной работы	43
6.5	Содержание отчета	44

Лабораторная работа 1

Исследование процессов обнаружения, сопряжения и установления соединения в сетях Bluetooth

1.1 Цель работы

Получение базовых теоретических знаний о механизмах работы сетей Bluetooth, а также практических навыков управления процессами поиска (inquiry), сопряжения (pairing) и установления соединения (connection) с использованием официального стека BlueZ в ОС семейства Linux.

1.2 Задачи

- Изучить теоретические основы процедур Inquiry, Page, Pairing и Connection в архитектуре Bluetooth BR/EDR.
- Ознакомиться с утилитами управления стеком BlueZ и анализатором трафика btmon.
- На практике выполнить сканирование окружающих Bluetooth-устройств, зафиксировать их адреса и классы устройств.
- Выполнить процедуру сопряжения с заданным устройством.
- Установить активное логическое соединение и проверить его состояние.
- Протоколировать этапы работы для оформления отчета.

1.3 Теоретический материал

1.3.1 Процесс обнаружения устройств (Inquiry и Page)

В сетях Bluetooth обнаружение устройств состоит из двух последовательных фаз: **Inquiry** (поиск) и **Page** (вызов).

- **Inquiry scan**: Устройства, находящиеся в режиме обнаружения, периодически прослушивают эфир на специальных частотах (Inquiry hopping sequence).
- **Inquiry**: Иницилирующее устройство (Inquirer) рассылает ID-пакеты по частотной сетке поиска. Устройства в режиме сканирования отвечают пакетом **FHS (Frequency Hop Synchronization)**, который содержит 48-битный адрес устройства (BD_ADDR), тип часов, класс устройства (CoD) и параметры синхронизации.

После получения FHS-пакетов инициатор формирует список найденных устройств. Для перехода к следующему этапу используется процедура **Page**, в ходе которой устройства синхронизируются по часам и устанавливают канал управления (LMP — Link Manager Protocol).

1.3.2 Процесс сопряжения (Pairing и Bonding)

Сопряжение — это процесс безопасного обмена ключами между двумя устройствами для установления защищённого канала.

- **Pairing** (сопряжение): процедура обмена аутентификационными ключами. В современных спецификациях (версия 2.1+) используется механизм **SSP (Secure Simple Pairing)**, основанный на алгоритме Диффи-Хеллмана (ECDH). SSP поддерживает несколько моделей ассоциации: Numeric Comparison, Just Works, Passkey Entry и Out of Band.
- **Bonding** (связывание): сохранение сгенерированного **Link Key** в энергонезависимой памяти устройств. После связывания последующие соединения могут устанавливаться автоматически без повторного прохождения процедуры аутентификации.

На уровне HCI (Host Controller Interface) процедура сопряжения реализуется через обмен специфическими пакетами команд и событий между хостом (ОС) и Bluetooth-контроллером. Ключевые этапы сопровождаются следующими пакетами:

- **IO Capability Request/Reply** (HCI Event 0x31 / HCI Command 0x042b): устройства обмениваются информацией о возможностях ввода-вывода для выбора оптимального метода ассоциации SSP.
- **User Confirmation Request** (HCI Event 0x33) и **Reply** (HCI Command 0x042c): используются в режиме Numeric Comparison для подтверждения пользователем совпадения 6-значного кода на обоих устройствах.
- **Link Key Request** (HCI Event 0x17) / **Link Key Negative Reply** (HCI Command 0x040c): контроллер запрашивает у хоста сохранённый ключ связи; при первом подключении хост возвращает отрицательный ответ, что инициирует генерацию нового ключа.
- **Link Key Notification** (HCI Event 0x18): контроллер передаёт хосту вновь сгенерированный Link Key для сохранения в базе данных сопряжённых устройств.
- **Encryption Change** (HCI Event 0x08): событие, подтверждающее активацию аппаратного шифрования на уровне контроллера после успешной аутентификации.

В случае успешного сопряжения активируется шифрование на уровне контроллера (Baseband), обеспечивающее конфиденциальность передаваемых данных. Анализ HCI-пакетов в утилитах `btmon` или Wireshark позволяет отладить процедуру сопряжения, определить выбранный метод SSP и проконтролировать корректность установки защищённого канала.

1.3.3 Установление соединения (Connection)

После успешного Page/Pairing на уровне хоста (Host) создаётся логический канал через подуровень **L2CAP (Logical Link Control and Adaptation Protocol)**. L2CAP обеспечивает мультиплексирование нескольких протоколов более высокого уровня над одним физическим каналом. Для установления соединения используются профили (например, **RFCOMM** для эмуляции последовательного порта, **A2DP** для аудио, **HID** для ввода). Клиент запрашивает соединение у серверного процесса, указывая PSM (Protocol/Service Multiplexer). После установки L2CAP-соединения устройства могут обмениваться полезными данными в рамках выбранного профиля.

1.3.4 Стек BlueZ и основные инструменты управления

BlueZ — официальный стек протоколов Bluetooth для ядра Linux. Основной пользовательский интерфейс взаимодействия со стеком предоставляется через интерактивную утилиту `bluetoothctl`. Управление осуществляется через D-Bus API демона `bluetoothd`.

Таблица 1.1 — Основные команды утилиты `bluetoothctl`

Команда	Описание
<code>power on</code>	Включение адаптера Bluetooth
<code>scan on</code>	Запуск режима сканирования (Inquiry)
<code>scan off</code>	Остановка сканирования
<code>devices</code>	Вывод списка обнаруженных или сопряжённых устройств
<code>pair <BD_ADDR></code>	Запуск процедуры сопряжения с указанным устройством
<code>trust <BD_ADDR></code>	Добавление устройства в список доверенных (автосопряжение)
<code>connect <BD_ADDR></code>	Установление логического соединения
<code>info <BD_ADDR></code>	Вывод подробной информации об устройстве (ключи, профили, статус)
<code>quit / exit</code>	Завершение сеанса работы с утилитой

Для мониторинга низкоуровневого трафика (HCI-команды, события, ACL-пакеты) в пакете BlueZ предусмотрен анализатор `btmon`, аналогичный по назначению утилите `tcpdump` в IP-сетях.

1.4 Порядок выполнения лабораторной работы

Примечание. Для выполнения работы требуется наличие активного Bluetooth-адаптера на лабораторном ПК. Все команды, кроме `info` и `devices`, требуют прав суперпользователя или работы в группе `bluetooth`.

1. Запустите терминал и откройте утилиту управления стеком:

```
user@host:~$ sudo bluetoothctl
```

В результате откроется интерактивная оболочка с приглашением `[bluetooth] #`.

2. Проверьте состояние адаптера и включите его при необходимости:

```
[bluetooth]# show  
[bluetooth]# power on
```

Зафиксируйте вывод команды `show` (адрес адаптера, имя, текущее состояние).

3. Запустите процесс сканирования устройств (Inquiry):

```
[bluetooth]# scan on
```

Подождите 10–15 секунд. В терминале будут появляться сообщения вида `Device XX:XX:XX:XX:XX:XX`. После получения достаточного количества устройств выполните:

```
[bluetooth]# scan off
```

4. Выберите одно из обнаруженных устройств для дальнейшей работы. Выведите список найденных устройств и запомните его `BD_ADDR`:

```
[bluetooth]# devices
```

5. Запустите параллельный терминал и начните захват низкоуровневого трафика с помощью `btmon`:

```
user@host:~$ sudo btmon -w bt_trace.log
```

Утилита будет записывать сырые HCI-события в файл `bt_trace.log` в бинарном формате.

6. В первом терминале иницилируйте процедуру сопряжения (Pairing):

```
[bluetooth]# pair <BD_ADDR>
```

Следуйте инструкциям утилиты (при необходимости подтвердите PIN-код или код числового сравнения на обоих устройствах). Дождитесь сообщения `Pairing successful`.

7. Установите логическое соединение (Connection):

```
[bluetooth]# connect <BD_ADDR>
```

Убедитесь в выводе `Connection successful`. Выполните команду `info <BD_ADDR>` и зафиксируйте статус соединения, список доступных UUID и состояние шифрования.

8. Завершите работу с `bluetoothctl` и остановите `btmon` (нажатие `Ctrl-C`). Просмотрите захваченный журнал:

```
user@host:~$ btmon -r bt_trace.log
```

Найдите в логе этапы: Inquiry, Create Connection (Page), User Confirmation Request/Link Key, Encryption Change.

9. Выведите утилиту из интерактивного режима:

```
[bluetooth]# disconnect <BD_ADDR>  
[bluetooth]# quit
```

1.5 Содержание отчета

1. Заголовок согласно приложению.
2. Цель работы.
3. Задание согласно варианту.
4. Листинги результатов работы:
 - a) вывод команды `show` до и после включения адаптера;
 - b) список обнаруженных устройств (`devices`) с указанием `BD_ADDR` и класса устройства;
 - c) вывод команд `pair`, `connect`, `info` с комментариями к каждому этапу;
 - d) фрагменты лога `btmon`, иллюстрирующие пакеты Inquiry, Page и установку шифрования;
 - e) скриншоты или текстовые подтверждения статуса соединения.
5. Ответы на контрольные вопросы.
6. Краткие выводы по работе.

Примечание. Перед каждым листингом должна быть указана выполненная команда. Вывод `btmon` рекомендуется прилагать отдельным файлом или в виде вырезок, подтверждающих прохождение ключевых фаз протокола.

Контрольные вопросы

1. Опишите различие между фазами Inquiry и Page. Какие пакеты используются на каждом этапе?
2. Что такое FHS-пакет и какую информацию он передаёт инициатору поиска?
3. В чём заключается разница между процессами Pairing и Bonding в спецификации Bluetooth?

4. Какие модели ассоциации предусмотрены в механизме Secure Simple Pairing (SSP) и в каких сценариях они применяются?
5. Какую роль играет подуровень L2CAP при установлении соединения между хостами?
6. Как с помощью утилиты `btmon` отфильтровать только события, связанные с шифрованием канала?
7. Почему для работы с адаптером в BlueZ часто требуются права суперпользователя или добавление пользователя в группу `bluetooth`?

Лабораторная работа 2

Исследование архитектуры пикосети Bluetooth и организация обмена данными через RFCOMM

2.1 Цель работы

Изучить принципы построения Bluetooth-пикосетей, технологию скачкообразной перестройки рабочей частоты (FHSS) и механизмы установления логических соединений. Получить практические навыки программирования сетевого взаимодействия в архитектуре «мастер–ведомые» с использованием Python, стека BlueZ и среды разработки Arduino IDE.

2.2 Задачи

- Изучить топологию пикосети, роли устройств (мастер/слейв) и ограничения архитектуры.
- Рассмотреть принцип работы FHSS, структуру частотной сетки и синхронизацию каналов.
- Ознакомиться с протоколом RFCOMM, его назначением и особенностями эмуляции последовательного порта.
- Освоить процесс подготовки среды Arduino IDE для работы с ESP32 и загрузки прошивки.
- Модифицировать базовую прошивку узла.
- Настроить два модуля ESP32 в режиме Bluetooth Classic SPP-серверов.
- Разработать и запустить Python-скрипт на ПК, устанавливающий параллельные RFCOMM-соединения с обоими устройствами.
- Организовать двунаправленный обмен данными, проверить наличие ФИО в возвращаемых пакетах.

2.3 Теоретический материал

2.3.1 Архитектура пикосети и роли устройств

Базовой единицей любой сети Bluetooth является **пикосеть** (piconet). Она представляет собой логическую сеть, образованную одним ведущим устройством (**мастером**) и до семи активных ведомых устройств (**слейвов**). Дополнительно в пикосети могут находиться до 255 устройств в режиме ожидания

(паркованные устройства, parked), не участвующие в активном обмене данными, но готовых к активации.

Мастер выполняет функции синхронизации: его внутренние часы и 48-битный Bluetooth-адрес (BD_ADDR) используются для генерации псевдослучайной частотной последовательности и определения временных слотов. Все слейвы синхронизируются по часам мастера и следуют единому расписанию скачков частоты. В рамках одной пикосети обмен данными возможен только между мастером и слейвами; прямое взаимодействие слейв–слейв не поддерживается на физическом уровне.

2.3.2 Технология FHSS и организация физического канала

Для обеспечения помехоустойчивости и безопасности в сетях Bluetooth BR/EDR применяется технология **FHSS (Frequency Hopping Spread Spectrum)**. Рабочий диапазон 2,4–2,4835 ГГц делится на 79 каналов с шагом 1 МГц (в некоторых регионах — 23 или 25 каналов). Скорость скачков составляет 1600 переходов в секунду (в базовом режиме BR).

Последовательность каналов вычисляется по алгоритму, использующему 28-битное значение адреса мастера, 28-битные внутренние часы и 1-битный идентификатор режима. Таким образом, каждое устройство в пикосети «знает», на какой частоте будет находиться мастер в следующий временной слот. Параметры физического канала представлены в табл. 2.1.

Таблица 2.1 — Параметры физического канала и технологии FHSS

Параметр	Значение / Описание
Частотный диапазон	2400–2483,5 МГц (ISM-диапазон)
Количество каналов	79 (шаг 1 МГц)
Скорость скачков	1600 гоп/с (1,6 кГц)
Время удержания частоты	625 мкс (1 слот)
Синхронизация	По часам мастера и его BD_ADDR
Модуляция	GFSK (Gaussian Frequency-Shift Keying)

Организация канала на уровне контроллера включает: *физический канал* (определяется частотной сеткой), *логический транспортный канал* (асинхронный ACL или синхронный SCO), *логический канал* (для конкретного соедине-

ния) и *логический транспортный канал* (каналы данных и сигнализации). Мастер управляет распределением временных слотов между слейвами, используя пакетную передачу данных.

2.3.3 Протокол RFCOMM и программная реализация

Для передачи потоковых данных в архитектуре Bluetooth используется подуровень **RFCOMM**, эмулирующий последовательный порт (эмуляция RS-232). RFCOMM работает поверх L2CAP и использует концепцию *Data Link Connection Identifiers* (DLCI) для мультиплексирования до 60 виртуальных последовательных соединений в рамках одного физического канала.

В ОС семейства Linux реализация RFCOMM доступна через системные сокеты. В языке Python для работы с Bluetooth Classic и установления RFCOMM-соединений используется стандартная библиотека `socket` с доменом `AF_BLUETOOTH` и протоколом `BTPROTO_RFCOMM`. После установления соединения программный интерфейс полностью идентичен работе с обычными TCP-сокетами, что упрощает разработку клиент-серверных приложений для обмена текстовыми и бинарными данными.

2.3.4 Среда разработки Arduino IDE и загрузка прошивки в ESP32

Для программирования микроконтроллеров ESP32 в данной лабораторной работе используется среда **Arduino IDE**. Это кроссплатформенное приложение, позволяющее писать код на C/C++ и загружать его в микроконтроллер через USB-интерфейс.

Основные этапы подготовки и загрузки:

1. **Установка поддержки ESP32:** В настройках Arduino IDE (File → Preferences) в поле *Additional Boards Manager URLs* необходимо добавить ссылку на репозиторий ESP32: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json. Затем в Board Manager устанавливается пакет *esp32 by Espressif Systems*.
2. **Выбор платы и порта:** В меню Tools → Board выбирается конкретная модель (например, *ESP32 Dev Module*). В меню Tools → Port выбира-

ется соответствующий USB-порт (в Linux обычно `/dev/ttyUSB0` или `/dev/ttyACM0`).

3. **Права доступа в Linux:** Для загрузки прошивки пользователь должен иметь права на запись в USB-порт. Обычно это решается добавлением пользователя в группу `dialout`:

```
user@host:~$ sudo usermod -a -G dialout $USER
```

После выполнения команды требуется перезагрузка системы или выход из сеанса.

4. **Компиляция и загрузка:** Код компилируется и загружается нажатием кнопки *Upload* (стрелка вправо). При загрузке модуль ESP32 автоматически переходит в режим программирования (благодаря схеме автопереключения на базе транзисторов или ручному нажатию кнопки BOOT).

2.4 Порядок выполнения лабораторной работы

Примечание. Для выполнения работы требуется ПК с установленным Linux и стеком BlueZ, а также два микроконтроллера ESP32 с поддержкой Bluetooth Classic. Перед началом работы устройства должны быть сопряжены с ПК (см. Лабораторную работу 1).

1. Модификация прошивки (идентификация студента):

- Откройте предоставленный базовый скетч для ESP32 в Arduino IDE.
- Найдите в коде функцию обработки входящих сообщений (блок `if (SerialBT.available())`).
- В месте формирования ответного сообщения найдите переменную, отвечающую за статус или текст ответа (например, `String response = "YOUR MESSAGE";` или аналогичную константу).
- Замените стандартное значение этой переменной на ваше **Фамилию Имя Отчество**

2. **Загрузка прошивки:** Подключите первый модуль ESP32 к ПК, выберите правильный порт и плату в Arduino IDE, затем загрузите модифицированную прошивку. Повторите процедуру для второго модуля. Запишите 48-битные MAC-адреса обоих устройств.

3. Проверка и освобождение каналов связи:

- Linux (BlueZ) может автоматически устанавливать соединение с ранее сопряженными устройствами при их включении. Это заблокирует порт для вашего Python-скрипта.

- Проверьте статус подключений командой:

```
user@host:~$ bluetoothctl devices connected
```

- Если ваши устройства (ESP32-Node1/Node2) отображаются в списке подключенных, необходимо принудительно разорвать соединение для каждого из них:

```
user@host:~$ bluetoothctl disconnect XX:XX:XX:XX:XX:XX
```

- Убедитесь, что устройства остаются в состоянии `paired` (сопряжены), но `not connected` (не подключены).

4. Установите необходимые системные пакеты на лабораторном ПК (если не установлены):

```
user@host:~$ sudo apt update
user@host:~$ sudo apt install python3 python3-pip libbluetooth-
dev
user@host:~$ pip3 install --user pybluez
```

5. Создайте файл `bt_master.py` и реализуйте скрипт, устанавливающий два параллельных RFCOMM-соединения. Скрипт должен:

- Подключаться к обоим устройствам.
- Отправлять тестовые команды (например, "PING").
- Принимать ответы и проверять, содержится ли в них ваше ФИО.

Базовая структура приведена в листинге 1. Замените `MAC_ESP1` и `MAC_ESP2` на реальные адреса ваших модулей.

6. Запустите скрипт с правами суперпользователя (для доступа к HCI-интерфейсу):

```
user@host:~$ sudo python3 bt_master.py
```

7. Наблюдайте за выводом терминала. Скрипт должен подключиться к устройствам, отправить команду и получить ответ вида `[ACK-NODE2] Received: PING | Status: PONG`.

8. В параллельном окне терминала запустите мониторинг трафика:

```
user@host:~$ sudo btmon -w lab02_trace.log
```

После завершения обмена остановите `btmon` и просмотрите лог, обращая внимание на этапы `L2CAP Connect Request`, `RFCOMM SABM` и `Encryption Change`.

9. Зафиксируйте в отчёте успешные подключения, факт возврата правильного ФИО от обоих устройств, переданные/принятые байты.

Listing 1: Базовая структура Python-скрипта для управления пикосетью

```
import socket
import time
import bluetooth

# MAC addresses of slave devices (replace with real ones)
ESP1_MAC = "AA:BB:CC:DD:EE:01"
ESP2_MAC = "AA:BB:CC:DD:EE:02"
def connect_rfcomm(mac_addr, port=RFCOMM_PORT):
    sock = socket.socket(socket.AF_BLUETOOTH,
                        socket.SOCK_STREAM,
                        socket.BTPROTO_RFCOMM)

    try:
        sock.connect((mac_addr, port))
        print(f"[+] Connected to {mac_addr}")
        return sock
    except Exception as e:
        print(f"[-] Connection error to {mac_addr}: {e}")
        return None

def exchange_data(sock, device_id):
    if not sock: return
    #msg = f"Data from Master -> {device_id}\n"
    msg = "PING"
    sock.send(msg.encode('utf-8'))
    response = sock.recv(1024).decode('utf-8').strip()
    print(f"<- Response from {device_id}: {response}")

def main():
    print("=== Piconet initialization ===")
    s1 = connect_rfcomm(ESP1_MAC)
    s2 = connect_rfcomm(ESP2_MAC)
```

```

if s1 and s2:
    print("=== Both channels active. Starting exchange ===")
    for i in range(5):
        exchange_data(s1, "ESP32_1")
        exchange_data(s2, "ESP32_2")
        time.sleep(1)

    s1.close()
    s2.close()
    print("=== Connections closed ===")
else:
    print("[-] Failed to establish all connections")

if __name__ == "__main__":
    main()

```

2.5 Содержание отчета

1. Заголовок согласно приложению.
2. Цель работы.
3. Задание согласно варианту.
4. Листинги результатов работы:
 - a) схема топологии созданной пикосети с указанием ролей и MAC-адресов;
 - b) фрагмент кода прошивки ESP32 с указанием места вставки ФИО;
 - c) листинг Python-скрипта с комментариями;
 - d) вывод терминала с результатами подключения и подтверждением наличия ФИО в ответе устройства;
 - e) фрагменты лога btmon, подтверждающие установление L2CAP- и RFCOMM-каналов;
5. Ответы на контрольные вопросы.
6. Краткие выводы по работе.

Примечание. При оформлении отчёта допускается сокращать листинги до ключевых фрагментов. Вывод btmon рекомендуется прилагать в виде вырезок, иллюстрирующих этапы установки соединения и передачи данных.

Контрольные вопросы

1. Что такое пикосеть и каковы её основные ограничения по количеству активных устройств?
2. Какую роль выполняет мастер в организации пикосети? Чем его функции отличаются от функций слейва?
3. Объясните принцип работы технологии FHSS. Почему скорость скачков в Bluetooth составляет именно 1600 гоп/с?
4. Как адрес мастера и его внутренние часы влияют на генерацию частотной последовательности?
5. Какую задачу решает протокол RFCOMM в стеке Bluetooth и как он взаимодействует с L2CAP?
6. Почему при программировании сокетов в Python используется константа `BTPROTO_RFCOMM`?
7. Каким образом обеспечивается изоляция логических каналов в рамках одного физического соединения?
8. Какие этапы установки RFCOMM-соединения можно наблюдать в логе `btmon`?
9. Какая группа пользователей в Linux необходима для доступа к последовательным портам при прошивке ESP32?

Лабораторная работа 3

Анализ трафика Bluetooth с использованием Wireshark

3.1 Цель работы

Получить практические навыки захвата и анализа сетевого трафика Bluetooth с использованием программного анализатора протоколов Wireshark. Изучить структуру пакетов Bluetooth, процедуры обнаружения устройств, сопряжения и обмена данными.

3.2 Задачи

- Изучить принципы работы анализаторов протоколов и возможности Wireshark для анализа Bluetooth трафика.
- Настроить систему для захвата Bluetooth пакетов.
- Захватить трафик процедур обнаружения (Inquiry), подключения (Connection) и сопряжения (Pairing) устройств.
- Проанализировать структуру захваченных пакетов: заголовки, типы пакетов, поля управления.
- Исследовать трафик различных профилей Bluetooth (SPP, A2DP, HID).
- Выполнить декодирование и интерпретацию содержимого пакетов.

3.3 Теоретический материал

3.3.1 Анализаторы протоколов

Анализатор протоколов (сниффер) — это программное или аппаратное средство, предназначенное для перехвата, декодирования и анализа сетевого трафика. Wireshark является одним из наиболее популярных открытых анализаторов протоколов, поддерживающим сотни протоколов, включая Bluetooth.

Основные возможности Wireshark:

- Захват пакетов в реальном времени с различных интерфейсов;
- Детальное декодирование протоколов с иерархическим отображением полей;
- Фильтрация трафика по различным критериям;
- Статистический анализ трафика;
- Экспорт данных в различные форматы.

3.3.2 Захват Bluetooth трафика

Для захвата Bluetooth трафика в Wireshark требуется:

1. **Адаптер Bluetooth** с поддержкой режима promiscuous mode (смешанного режима);
2. **Драйверы**, поддерживающие захват пакетов (в Linux — btmon, в Windows — USBPcap для USB-адаптеров);
3. **Права суперпользователя** для доступа к низкоуровневым интерфейсам.

Wireshark может захватывать Bluetooth трафик несколькими способами:

- **Linux Sockets**: через интерфейс Bluetooth в Linux;
- **USBPcap**: для захвата USB-трафика Bluetooth-адаптера;
- **PCAP-файлы**: анализ предварительно захваченного трафика;
- **Специализированные снифферы**: использование внешних устройств (Ubertooth, Ellisys).

3.3.3 Структура пакетов Bluetooth

Пакеты Bluetooth состоят из нескольких уровней:

1. Физический уровень (Physical Layer):

- Access Code (68–72 бита) — код доступа;
- Header (54 бита) — заголовок пакета;
- Payload (0–2745 бит) — полезная нагрузка.

2. Канальный уровень (Link Layer): Управляет установлением соединений, повторной передачей и контролем потока.

3. Протоколы верхних уровней:

- **L2CAP** (Logical Link Control and Adaptation Protocol) — адаптация пакетов;
- **RFCOMM** — эмуляция последовательных портов;
- **SDP** (Service Discovery Protocol) — обнаружение сервисов;
- **ATT/GATT** — для Bluetooth Low Energy;
- **Профили** (A2DP, HID, HFP, SPP и др.).

3.3.4 Основные процедуры Bluetooth

Inquiry (Обнаружение устройств): Процедура поиска устройств в радиусе действия. Инициатор рассылает ID-пакеты, устройства в режиме обнаруже-

ния отвечают FHS-пакетами со своей информацией (BD_ADDR, класс устройства, часы).

Paging (Установление соединения): Процедура подключения к конкретному устройству по известному BD_ADDR. Включает синхронизацию по часам и частоте.

Pairing (Сопряжение): Процесс обмена ключами безопасности для аутентификации и шифрования. Может включать ввод PIN-кода или использование SSP (Secure Simple Pairing).

Service Discovery (Обнаружение сервисов): Запрос информации о доступных сервисах и профилях на удаленном устройстве через протокол SDP.

3.4 Порядок выполнения лабораторной работы

Примечание. Для выполнения работы требуется ПК с Bluetooth-адаптером, установленным Wireshark и правами суперпользователя.

1. Подготовка среды захвата:

- a) Установите Wireshark (если не установлен):

```
user@host:~$ sudo apt install wireshark
```

- b) Добавьте пользователя в группу wireshark для захвата пакетов без root:

```
user@host:~$ sudo usermod -a -G wireshark $USER
```

- c) Проверьте доступные интерфейсы для захвата:

```
user@host:~$ wireshark -D
```

Найдите Bluetooth-интерфейс (обычно bluetooth0 или btmon).

2. Захват трафика процедуры обнаружения (Inquiry):

- a) Запустите Wireshark от имени суперпользователя:

```
user@host:~$ sudo wireshark
```

- b) Выберите Bluetooth-интерфейс для захвата и начните захват.

- c) В другом терминале запустите сканирование Bluetooth-устройств:

```
user@host:~$ sudo bluetoothctl scan on
```

- d) Подождите 10–15 секунд, пока будут найдены устройства.

- e) Остановите захват в Wireshark.

- f) Сохраните файл захвата как inquiry.pcapng.

3. Анализ пакетов Inquiry:

- a) Примените фильтры для детального анализа процедуры обнаружения устройств. В строке фильтра Wireshark используйте следующие выражения:
 - `bthci_evt.code == 0x01` — **Inquiry Complete**. Фиксирует событие завершения процедуры поиска контроллером.
 - `bthci_evt.code == 0x02` — **Inquiry Result**. Основной фильтр: отображает пакеты с базовой информацией о найденных классических устройствах.
 - `bthci_evt.code == 0x2f` — **Extended Inquiry Result**. Отображает пакеты с расширенными данными (полное имя устройства EIR, класс устройства, RSSI).
- b) Найдите пакеты `Extended Inquiry Result` и проанализируйте их поля в нижней панели декодирования:
 - `BD_ADDR` обнаруженных устройств;
 - `Class of Device (CoD)`;
 - `RSSI` (уровень сигнала).
- c) Сделайте скриншот с развёрнутой структурой пакета `Extended Inquiry Result`, обратив внимание на поле *EIR Data* с именем устройства.
- d) Заполните таблицу найденных устройств (Табл. 3.1).

4. Захват трафика сопряжения (Pairing):

- a) Начните новый захват в Wireshark.
- b) Выполните сопряжение с Bluetooth-устройством (наушники, телефон и т.д.):

```
user@host:~$ bluetoothctl
[bluetooth]# pair XX:XX:XX:XX:XX:XX
```

- c) Подтвердите сопряжение (введите PIN-код, если требуется).
- d) После успешного сопряжения остановите захват.
- e) Сохраните файл как `pairing.pcapng`.

5. Анализ процедуры Pairing:

- a) Примените фильтр для HCI-событий сопряжения:

```
bthci_cmd.opcode == 0x0405 || bthci_evt.event_code == 0x06
```

- b) Найдите и проанализируйте пакеты:

- Link Key Request;
 - IO Capability Request/Response;
 - User Confirmation Request (если используется);
 - Link Key Notification.
- c) Определите метод сопряжения (Just Works, Passkey Entry, Numeric Comparison).
- d) Сделайте скриншоты ключевых этапов сопряжения.
- 6. Захват трафика передачи данных:**
- a) Начните новый захват.
- b) Отправьте файл на сопряженное устройство или получите файл с него:
- ```
user@host:~$ bluetoothctl connect XX:XX:XX:XX:XX:XX
user@host:~$ obexctl
```
- c) Альтернативно: подключите Bluetooth-наушники и воспроизведите аудио (A2DP трафик).
- d) Остановите захват после передачи данных.
- e) Сохраните как `data_transfer.pcapng`.
- 7. Анализ трафика передачи данных:**
- a) Примените фильтр для L2CAP-пакетов:
- ```
bt12cap
```
- b) Найдите пакеты RFCOMM (если использовался SPP):
- ```
btrfcomm
```
- c) Проанализируйте:
- Размер пакетов;
  - Интервалы между пакетами;
  - Количество повторных передач (retransmissions).
- d) Для A2DP-трафика найдите пакеты с аудио-данными.
- e) Сделайте скриншот статистики потока (Statistics → Conversations → Bluetooth).
- 8. Статистический анализ:**
- a) Откройте статистику захваченного трафика:
- Statistics → Protocol Hierarchy;
  - Statistics → Conversations → Bluetooth;
  - Statistics → IO Graphs.

- b) Постройте график интенсивности трафика во времени.
- c) Определите:
  - Общее количество пакетов;
  - Количество байт;
  - Среднюю скорость передачи;
  - Доминирующие протоколы.

Таблица 3.1 — Обнаруженные Bluetooth-устройства

| № | BD_ADDR | Имя устройства | устрой- | Class of Device | RSSI (дБм) |
|---|---------|----------------|---------|-----------------|------------|
| 1 |         |                |         |                 |            |
| 2 |         |                |         |                 |            |
| 3 |         |                |         |                 |            |

### 3.5 Содержание отчета

1. Заголовок согласно приложению.
2. Цель работы.
3. Краткое описание использованного оборудования (модель Bluetooth-адаптера).
4. Результаты работы:
  - a) таблица обнаруженных устройств (Табл. 3.1);
  - b) скриншоты Wireshark с анализом Inquiry-пакетов;
  - c) скриншоты этапов процедуры Pairing с пояснениями;
  - d) скриншоты трафика передачи данных с расшифровкой полей;
  - e) статистика трафика (Protocol Hierarchy, Conversations);
  - f) график интенсивности трафика (IO Graph).
5. Выводы по работе: какие протоколы и процедуры были проанализированы, какие особенности трафика выявлены.

**Примечание.** Все скриншоты должны быть читаемыми, с выделенными ключевыми полями пакетов. При необходимости используйте зум в Wireshark.

## Контрольные вопросы

1. Какие уровни OSI модели задействованы в стеке протоколов Bluetooth?
2. В чем разница между процедурами Inquiry и Paging?
3. Какие методы сопряжения (Pairing) существуют в Bluetooth? Какой из них наиболее безопасен?
4. Что такое BD\_ADDR и какова его структура?
5. Какую функцию выполняет протокол L2CAP?
6. Чем отличается трафик SPP от трафика A2DP?
7. Какие фильтры Wireshark можно использовать для анализа Bluetooth-трафика?
8. Что такое Class of Device (CoD) и как он кодируется?
9. Как в Wireshark определить, используется ли шифрование в Bluetooth-соединении?
10. Какие проблемы могут возникнуть при захвате Bluetooth-трафика и как их решить?

## Лабораторная работа 4

### Организация удаленного выполнения команд через Bluetooth-мост на базе ESP32

#### 4.1 Цель работы

Изучить принципы ретрансляции данных в сетях Bluetooth. Получить практические навыки создания многоточечных соединений с участием встроенных систем. Реализовать архитектуру «Клиент–Шлюз–Сервер», где микроконтроллер ESP32 выступает в роли прозрачного моста между двумя хостами, обеспечивая удаленное выполнение команд на целевом ПК.

#### 4.2 Задачи

- Изучить возможности одновременной работы ESP32 в режимах SPP-клиента и SPP-сервера.
- Рассмотреть концепцию туннелирования данных через промежуточный узел.
- Настроить два ПК: один как источник команд (Client), другой как исполнитель (Server).
- Разработать прошивку для ESP32, осуществляющую двунаправленную пересылку байтов между двумя Bluetooth-каналами.
- Реализовать серверную часть на Python на втором ПК, выполняющую полученные команды в оболочке ОС.
- Разработать клиентский терминал на первом ПК для отправки команд и отображения результатов.
- Протоколировать задержки и целостность данных при прохождении через промежуточный узел.

#### 4.3 Теоретический материал

##### 4.3.1 Архитектура «Bluetooth-мост»

В классической пикосети Bluetooth одно устройство является мастером, а другие — слейвами. Однако современные чипы, такие как ESP32, поддерживают роль **Dual Mode** и могут одновременно поддерживать несколько активных соединений или выступать в разных ролях для разных устройств.



В данной работе реализуется топология, где ESP32:

1. Выступает в роли **SPP Server** для ПК-Клиента (принимает команды).
2. Выступает в роли **SPP Client** для ПК-Сервера (подключается к нему и передает команды).

Таким образом, ESP32 действует как аппаратный шлюз. Данные, поступившие от Клиента по каналу А, считываются микроконтроллером и немедленно передаются в канал В (на Сервер). Ответ от Сервера по каналу В считывается и перенаправляется в канал А (Клиенту).

#### 4.3.2 Проблемы буферизации и потокового управления

При ретрансляции данных через микроконтроллер критически важно учитывать разницу в скоростях обработки.

- **Буферизация:** Если ПК-Сервер обрабатывает команду долго (например, компиляция или тяжелый запрос к БД), ESP32 должен хранить входящие данные во внутреннем буфере или приостанавливать прием (Flow Control), чтобы избежать переполнения памяти.
- **Синхронность:** В простейшей реализации используется блокирующий ввод-вывод. ESP32 ждет завершения приема пакета от Клиента, затем ждет отправки на Сервер, затем ждет ответа. Это создает суммарную задержку  $T_{total} = T_{BT1} + T_{proc} + T_{BT2}$ .

#### 4.3.3 Удаленное выполнение команд (Remote Code Execution)

Для выполнения команд на ПК-Сервере используется механизм вызова системной оболочки (shell). В Python за это отвечает модуль `subprocess`.

```
import subprocess
result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
output = result.stdout + result.stderr
```

**Внимание!** Открытый доступ к выполнению произвольных команд представляет угрозу безопасности. В рамках лабораторной работы предполагается использование доверенной локальной сети.

## 4.4 Порядок выполнения лабораторной работы

### Оборудование:

- ПК 1 (Client): Отправитель команд.
- ПК 2 (Server): Исполнитель команд (должен иметь Bluetooth-адаптер).
- Модуль ESP32.

### 1. Подготовка ПК-Сервера (Target):

- На ПК 2 создайте Python-скрипт `bt_server.py`, который:
  - a) Запускает SPP-сервер (через `rfcomm` или прямой сокет).
  - b) Ожидает подключения от ESP32.
  - c) При получении строки выполняет её через `subprocess`.
  - d) Возвращает `stdout/stderr` обратно в Bluetooth-канал.
- Запишите MAC-адрес Bluetooth-адаптера ПК 2.

### 2. Прошивка ESP32 (Bridge):

- Используйте код из Листинга 4.1.
- В коде замените `SERVER_MAC` на MAC-адрес ПК 2.
- Загрузите прошивку в ESP32.

### 3. Настройка ПК-Клиента (Source):

- Сопрягите ПК 1 с ESP32.
- Привяжите ESP32 к порту: `sudo rfcomm bind /dev/rfcomm0 <MAC_ESP32>`.
- Используйте скрипт `bt_client.py` (Листинг 4.2) для отправки команд.

### 4. Запуск системы:

- a) На ПК 2 запустите `python3 bt_server.py`. Он перейдет в режим ожидания подключения.
- b) Перезагрузите ESP32. Он автоматически подключится к ПК 2 и начнет ждать подключения от ПК 1.
- c) На ПК 1 запустите `python3 bt_client.py`.

### 5. Тестирование:

- Отправьте команду `uname -a`. Убедитесь, что вернулась информация о ядре ПК 2.

- Отправьте команду `ls -la /tmp`. Проверьте вывод списка файлов.
- Отправьте некорректную команду (например, `fake_cmd`). Убедитесь, что вернулась ошибка (`stderr`).
- Измерьте время отклика для простых команд (`echo test`) и сложных (`ping -c 1 google.com`).

## 6. Анализ трафика:

- Запустите `btmon` на ПК 1 и ПК 2 одновременно.
- Сравните временные метки пакетов ACL для выявления задержек на этапе ретрансляции в ESP32.

*Listing 2: Листинг 4.2. Серверная часть на ПК 2 (Executor)*

```
import serial
import subprocess
import sys

PORT = '/dev/rfcomm0' # Or direct socket if rfcomm bind is not used
BAUDRATE = 115200

def execute_command(cmd):
 try:
 # Execute command in bash shell
 result = subprocess.run(
 cmd,
 shell=True,
 capture_output=True,
 text=True,
 timeout=10 # Execution timeout 10 sec
)
 output = result.stdout
 if result.stderr:
 output += "\n[ERR]: " + result.stderr
 return output.strip()
 except Exception as e:
 return f"[EXEC ERROR]: {str(e)}"

def main():
 print(f"Listening on {PORT}...")
 try:
 ser = serial.Serial(PORT, BAUDRATE, timeout=None)
 while True:
```

```

 # Read command until newline
 cmd_bytes = ser.readline()
 if not cmd_bytes:
 continue

 cmd = cmd_bytes.decode('utf-8').strip()
 print(f"Received: {cmd}")

 if cmd.lower() == 'exit':
 break

 # Execute and get result
 response = execute_command(cmd)

 # Send result back
 ser.write((response + "\n").encode('utf-8'))

 except Exception as e:
 print(f"Error: {e}")
 finally:
 if 'ser' in locals():
 ser.close()

if __name__ == "__main__":
 main()

```

## 4.5 Содержание отчета

1. Заголовок согласно приложению.
2. Цель работы.
3. Схема соединения (ПК1 ↔ ESP32 ↔ ПК2).
4. Листинги результатов работы:
  - a) код прошивки ESP32 с комментариями по логике моста;
  - b) код серверной части на Python;
  - c) лог работы клиентского терминала (примеры успешных и ошибочных команд);
  - d) таблица замеров времени отклика (RTT) для различных типов команд.
5. Анализ задержек: оценка вклада каждого звена цепи в общую задержку.
6. Ответы на контрольные вопросы.

7. Выводы о надежности такой схемы для задач удаленного администрирования.

### **Контрольные вопросы**

1. Какие ограничения накладывает архитектура ESP32 на количество одновременных Bluetooth-соединений?
2. Почему при ретрансляции данных через микроконтроллер может возникать проблема рассинхронизации потоков?
3. Как обеспечить безопасность при удаленном выполнении команд (Remote Code Execution)?
4. Что произойдет с системой, если связь между ESP32 и ПК-Сервером оборвется во время выполнения длительной команды?
5. Чем отличается данная схема от прямого SSH-подключения к ПК-Серверу? В каких случаях оправдано использование промежуточного Bluetooth-узла?
6. Как влияет размер буфера UART в ESP32 на максимальную длину передаваемой команды или ответа?

## **Лабораторная работа 5**

### **Исследование архитектуры Bluetooth Low Energy (BLE) и реализация кастомных профилей GATT**

#### **5.1 Цель работы**

Изучить архитектуру стека протоколов Bluetooth Low Energy (BLE), отличия от классического Bluetooth и структуру профиля Generic Attribute Profile (GATT). Получить практические навыки разработки кастомных BLE-сервисов и характеристик на базе микроконтроллеров ESP32, а также организации взаимодействия между двумя устройствами в топологии «Центральное–Периферийное».

#### **5.2 Задачи**

- Изучить теоретические основы BLE: роли устройств (Central/Peripheral), рекламные пакеты (Advertising), соединение и структуру GATT.
- Рассмотреть понятие UUID, сервисов (Services) и характеристик (Characteristics) в контексте BLE.
- Разработать прошивку для ESP32, реализующую роль BLE-периферии (Server) с кастомным сервисом.
- Разработать прошивку для второго ESP32, реализующего роль BLE-централи (Client) для подключения и чтения/записи данных.
- Организовать двунаправленный обмен данными через характеристики с правами READ, WRITE и NOTIFY.
- Протоколировать процесс соединения и обмена данными, оценить энергоэффективность по сравнению с классическим Bluetooth.

#### **5.3 Теоретический материал**

##### **5.3.1 Архитектура Bluetooth Low Energy (BLE)**

Bluetooth Low Energy (BLE, версия Bluetooth 4.0 и выше) был разработан с упором на сверхнизкое энергопотребление, что позволяет устройствам работать от одной батарейки типа «таблетка» в течение месяцев или лет. В отличие от классического Bluetooth, который ориентирован на постоянный поток

данных (аудио, файлы), BLE оптимизирован для передачи небольших пакетов данных с большими интервалами.

Ключевые отличия BLE от Classic Bluetooth:

- **Модуляция и каналы:** BLE использует 40 каналов шириной 2 МГц в диапазоне 2.4 ГГц (3 рекламных канала и 37 каналов данных). Классический Bluetooth использует 79 каналов по 1 МГц.
- **Роли устройств:** В BLE устройства делятся на **Broadcaster/Observer** (для рекламы без соединения) и **Peripheral/Central**. Peripheral (периферия) рекламирует себя и ждет подключения, Central (центральное устройство) сканирует эфир и инициирует соединение.
- **Скорость соединения:** BLE устанавливает соединение за несколько миллисекунд, тогда как классическому Bluetooth требуются секунды.
- **Потребление энергии:** BLE потребляет в 10–100 раз меньше энергии благодаря коротким импульсам передачи и длительным периодам сна.

### 5.3.2 Профили классического Bluetooth (BR/EDR)

Профили Bluetooth представляют собой стандартизированные спецификации, определяющие поведение устройств при реализации конкретных сценариев использования. Каждый профиль описывает, какие протоколы стека должны быть задействованы, как происходит обнаружение сервисов, параметры безопасности, роли устройств (инициатор/ответчик, сервер/клиент) и формат передаваемых данных. Профили обеспечивают аппаратную и программную интероперабельность между устройствами различных производителей.

В классическом Bluetooth (BR/EDR) профили строятся поверх протоколов транспортного и прикладного уровней. Для передачи потоковых данных и эмуляции последовательных портов используется протокол **RFCOMM**, работающий поверх L2CAP. Для мультимедийных задач применяются специализированные протоколы **AVDTP** (Audio/Video Distribution Transport Protocol) и **AVCTP** (Audio/Video Control Transport Protocol). Ключевым механизмом обнаружения доступных профилей является протокол **SDP (Service Discovery Protocol)**, который позволяет клиенту запросить у сервера список зарегистрированных сервисов по их уникальным идентификаторам (UUID).

Основные профили классического Bluetooth, наиболее широко применяемые в современных устройствах, представлены в табл. 5.1.

Таблица 5.1 — Основные профили классического Bluetooth (BR/EDR)

| Профиль (UUID)                 | Назначение и используемый стек протоколов                                                                                                                                                                      |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>GAP</b> (0x1800)            | <i>Generic Access Profile</i> . Базовый профиль, определяющий процедуры обнаружения устройств, установления соединения, режимы безопасности и модели взаимодействия. Использует L2CAP.                         |
| <b>SDP</b> (0x0001)            | <i>Service Discovery Protocol</i> . Обеспечивает поиск и получение информации о доступных сервисах и их параметрах на удалённых устройствах. Работает поверх L2CAP.                                            |
| <b>SPP</b> (0x1101)            | <i>Serial Port Profile</i> . Эмулирует последовательный порт (RS-232) поверх радиоканала. Широко используется для терминального доступа, прошивки МК и передачи сырых данных. Стек: RFCOMM → L2CAP.            |
| <b>HFP</b> (0x111F)            | <i>Hands-Free Profile</i> . Профиль «громкой связи» для автомобильных гарнитур и наушников. Поддерживает голосовые вызовы, управление звонками и передачу аудио. Стек: RFCOMM (управление) + SCO/eSCO (аудио). |
| <b>A2DP</b> (0x110D)           | <i>Advanced Audio Distribution Profile</i> . Профиль распределения стереоаудио высокого качества (музыка, подкасты). Использует асинхронный транспорт. Стек: AVDTP → L2CAP.                                    |
| <b>AVRCP</b> (0x110E)          | <i>Audio/Video Remote Control Profile</i> . Профиль дистанционного управления мультимедиа (play/pause, next/prev, громкость). Стек: AVCTP → L2CAP.                                                             |
| <b>HID</b> (0x1124)            | <i>Human Interface Device Profile</i> . Профиль для клавиатур, мышей, геймпадов и сканеров. Обеспечивает низкую задержку ввода. Стек: HIDP → L2CAP.                                                            |
| <b>PAN</b> (0x1115/0x1116)     | <i>Personal Area Networking</i> . Организация локальной сети поверх Bluetooth (режимы NAP, PANU, GN). Позволяет устройствам обмениваться IP-пакетами. Стек: BNEP → L2CAP.                                      |
| <b>FTP/OPP</b> (0x1106/0x1105) | <i>File Transfer / Object Push Profile</i> . Профили передачи файлов и объектов (визитки, календари, изображения). Исторически использу-                                                                       |



### Механизм активации профиля:

1. **Discovery:** Иницилирующее устройство выполняет процедуру поиска (Inquiry) и получает BD\_ADDR целевого узла.
2. **SDP Query:** Клиент подключается к SDP-серверу удалённого устройства и запрашивает список поддерживаемых UUID.
3. **Channel Allocation:** На основе ответа SDP клиент определяет номер канала RFCOMM (или PSM для L2CAP-сервисов), на котором запущен целевой профиль.
4. **Connection:** Устанавливается логическое соединение, после чего устройства переходят в режим обмена данными согласно спецификации профиля.

Важно отметить, что один физический Bluetooth-адаптер может одновременно поддерживать несколько активных профилей (например, HFP для звонков и A2DP для музыки), однако ресурсы контроллера и пропускная способность канала распределяются динамически, что может влиять на качество сервисов при высокой нагрузке.

### 5.3.3 Профиль GATT (Generic Attribute Profile)

Взаимодействие приложений в BLE строится поверх протокола ATT (Attribute Protocol), который абстрагирован профилем GATT. GATT определяет иерархическую структуру данных, которую сервер (Peripheral) предоставляет клиенту (Central).

Иерархия GATT:

1. **Profile (Профиль):** Коллекция сервисов, определяющая поведение устройства (например, Heart Rate Profile).
2. **Service (Сервис):** Логическая группа данных. Каждый сервис имеет уникальный 16-битный или 128-битный UUID. Например, сервис «Батарея» или кастомный сервис «Управление светодиодом».
3. **Characteristic (Характеристика):** Основной контейнер данных. Характеристика содержит:
  - *UUID:* Уникальный идентификатор характеристики.
  - *Properties:* Права доступа (READ, WRITE, NOTIFY, INDICATE).
  - *Value:* Само значение данных (до 512 байт, обычно меньше).
  - *Descriptors:* Метаданные о характеристике (например, Client

Characteristic

Configuration Descriptor — CCCD, необходимый для включения уведомлений).

Таблица 5.2 — Основные свойства характеристик BLE

| Свойство | Описание                                                                                                           |
|----------|--------------------------------------------------------------------------------------------------------------------|
| READ     | Клиент может читать значение характеристики.                                                                       |
| WRITE    | Клиент может записывать новое значение.                                                                            |
| NOTIFY   | Сервер может отправлять обновления значения клиенту без подтверждения (Unacknowledged). Быстрее, но менее надежно. |
| INDICATE | Аналог NOTIFY, но требует подтверждения от клиента (Acknowledged). Надежнее, но медленнее.                         |

### 5.3.4 Процесс подключения и обмена данными

1. **Advertising:** Периферия рассылает рекламные пакеты, содержащие свое имя, UUID доступных сервисов и другие данные.
2. **Scanning:** Центральное устройство сканирует эфир и фильтрует устройства по UUID или имени.
3. **Connection:** Центр инициирует соединение. После этого рекламные пакеты прекращаются, и начинается обмен данными по каналам связи.
4. **Service Discovery:** Клиент запрашивает список сервисов и характеристик сервера (Discover Services/Characteristics).
5. **Data Exchange:** Клиент читает/пишет данные или подписывается на уведомления (Enable Notifications).

## 5.4 Порядок выполнения лабораторной работы

**Примечание.** Для выполнения работы требуются два модуля ESP32. Один будет выступать в роли BLE Server (Peripheral), другой — BLE Client (Central).

1. **Настройка и загрузка прошивки BLE Server (ESP32\_A):**

- Откройте предоставленный файл скетча `ble_server.ino` в среде Arduino IDE.
- В начале файла найдите макросы, определяющие имя устройства (`DEVICE_NAME`) и 128-битные UUID сервиса и характеристик.
- При необходимости измените имя устройства на уникальное (например, добавьте номер бригады или варианта), чтобы избежать конфликтов при сканировании в аудитории.
- Убедитесь, что в системе установлена библиотека `BLEDevice.h` (входит в стандартный пакет ядра ESP32).
- Подключите первый модуль ESP32 к ПК, выберите соответствующий COM-порт и модель платы, затем скомпилируйте и загрузите прошивку.

## 2. Настройка и загрузка прошивки BLE Client (ESP32\_B):

- Откройте предоставленный файл скетча `ble_client.ino`.
- Найдите в коде константу, задающую имя целевого сервера (например, `TARGET_SERVER_NAME`) или UUID сервиса для фильтрации.
- Внесите изменения, указав имя или UUID, совпадающие с заданными в прошивке сервера на шаге 1.
- При необходимости скорректируйте интервал отправки данных или текст тестового сообщения в соответствующих переменных.
- Подключите второй модуль ESP32, выберите порт/плату и загрузите модифицированную прошивку.

## 3. Тестирование взаимодействия:

- Откройте два окна Serial Monitor (для ESP32\_A и ESP32\_B) со скоростью 115200 бод.
- Перезагрузите оба устройства (аппаратно или программно).
- Убедитесь по логам, что Client выполнил сканирование, нашёл Server по имени/UUID и успешно подключился.
- Проверьте, что сообщения, отправляемые Client, появляются в логе Server.
- Проверьте, что Server отправляет подтверждения (ACK) или собственные уведомления, которые корректно отображаются в логе Client.

4. **Сравнение с Classic Bluetooth:** Оцените время установки соединения, размер служебных заголовков и энергопотребление (косвенно, по логам отладки) в BLE по сравнению с протоколом RFCOMM, изученным в Лабораторной работе 3.

## 5.5 Содержание отчета

1. Заголовок согласно приложению.
2. Цель работы.
3. Задание согласно варианту.
4. Листинги результатов работы:
  - а) листинг прошивки BLE Server с пояснением выбранных UUID и свойств характеристик;
  - б) листинг прошивки BLE Client с логикой подключения и подписки на уведомления;
  - в) логи Serial Monitor обоих устройств, демонстрирующие успешный обмен данными;
  - г) схема иерархии GATT созданного сервиса (дерево Service → Characteristics).
5. Ответы на контрольные вопросы.
6. Краткие выводы по работе.

## Контрольные вопросы

1. В чем заключаются основные архитектурные отличия BLE от классического Bluetooth?
2. Что такое GATT и какова его роль в стеке протоколов BLE?
3. Объясните разницу между ролями Central и Peripheral. Может ли одно устройство менять роль во времени?
4. Что такое UUID и почему важно использовать уникальные UUID для кастомных сервисов?
5. В чем разница между свойствами NOTIFY и INDICATE? Когда целесообразно использовать каждое из них?
6. Зачем нужен дескриптор CCCD (Client Characteristic Configuration Descriptor)?

7. Как происходит процесс обнаружения сервисов (Service Discovery) и почему он может занимать значительное время?
8. Какие ограничения накладывает BLE на размер передаваемых данных (MTU) и как их можно расширить?

## **Лабораторная работа 6**

### **Сравнение скоростных характеристик протоколов Bluetooth (BR, EDR, BLE)**

#### **6.1 Цель работы**

Провести экспериментальное сравнение реальной пропускной способности каналов связи, организованных с использованием технологий классического Bluetooth (Basic Rate и Enhanced Data Rate) и Bluetooth Low Energy. Изучить влияние параметров модуляции, размера пакетов и интервалов соединения на итоговую скорость передачи данных.

#### **6.2 Задачи**

- Изучить теоретические основы физических уровней Bluetooth BR, EDR и BLE: схемы модуляции, скорость символа, структуру пакетов.
- Рассмотреть влияние параметров L2CAP MTU, интервалов соединения и режима подтверждения (ACK/No-ACK) на пропускную способность.
- Подготовить стенд из двух модулей ESP32 для проведения измерений: один в роли передатчика (TX), другой — приёмника (RX).
- Провести серию измерений пропускной способности для трёх режимов:
  - a) Classic Bluetooth (BR/EDR) с использованием прошивок `ESP32_BT_TX.ino` и `ESP32_BT_RX.ino`;
  - b) Bluetooth Low Energy (BLE) с использованием прошивок `ESP32_BLE_TX.ino` и `ESP32_BLE_RX.ino`.
- Рассчитать среднюю скорость передачи в кбит/с и КБайт/с, построить сравнительные графики.
- Проанализировать полученные результаты и сформулировать выводы о применимости каждой технологии для различных сценариев передачи данных.

#### **6.3 Теоретический материал**

##### **6.3.1 Физический уровень и модуляция**

Скорость передачи данных в Bluetooth определяется схемой модуляции и символьной скоростью на физическом уровне. Во всех режимах (BR, EDR и

BLE) базовая символьная скорость составляет 1 Мсимв/с, однако методы кодирования и итоговая пропускная способность существенно различаются.

Режим **Basic Rate (BR)**, соответствующий спецификации 1.x, использует гауссовскую частотную модуляцию (GFSK) с индексом модуляции 0.5. Это обеспечивает высокую помехоустойчивость при физической скорости передачи данных 1 Мбит/с. Типичная чувствительность современных приемников в этом режиме составляет около  $-90 \dots -95$  дБм, что позволяет достигать стабильной связи на расстоянии до 10–30 м в помещениях.

В спецификации **Enhanced Data Rate (EDR, версия 2.0 и выше)** для увеличения пропускной способности введены дополнительные схемы фазовой модуляции. В то время как служебный заголовок пакета всегда передается с использованием GFSK, поле полезной нагрузки кодируется методами  $\pi/4$ -DQPSK (2 бита на символ, до 2 Мбит/с) или 8DPSK (3 бита на символ, до 3 Мбит/с). Несмотря на рост скорости, EDR сохраняет сопоставимую с BR дальность работы, хотя и предъявляет более высокие требования к отношению сигнал/шум.

Протокол **Bluetooth Low Energy (BLE, версии 4.0 и выше)** оптимизирован для сверхнизкого энергопотребления. В версиях 4.0–4.2 используется режим 1M PHY (GFSK, 1 Мбит/с). Начиная со спецификации Bluetooth 5.0, возможности физического уровня были значительно расширены:

- **2M PHY:** Удваивает символьную скорость до 2 Мсимв/с, обеспечивая пиковую скорость передачи данных 2 Мбит/с.
- **Coded PHY (S=2 и S=8):** Вводит избыточное кодирование (FEC), что повышает чувствительность приемника до  $-100 \dots -105$  дБм. Это позволяет увеличить дальность связи до нескольких сотен метров (и более 1 км в условиях прямой видимости), снижая при этом полезную скорость до 500 кбит/с или 125 кбит/с соответственно.

Для повышения стабильности соединения в BLE часто применяется «стабильный индекс модуляции», варьирующийся в пределах  $0.45 \dots 0.55$ , что делает технологию оптимальной для критически важных узлов IoT.

### 6.3.2 Влияние протоколов верхних уровней на пропускную способность

Реальная скорость передачи полезных данных (throughput) всегда ниже сырой скорости физического уровня из-за служебных накладных расходов:

1. **Заголовки стека:** каждый пакет инкапсулируется в заголовки L2CAP, RFCOMM (для Classic) или ATT (для BLE), HCI, Baseband.
2. **Подтверждения (ACK):** в надёжных режимах (RFCOMM, BLE Indicate) передатчик ожидает ACK от приёмника, что вносит задержку и снижает эффективную скорость.
3. **Интервал соединения (Connection Interval):** в BLE устройства обмениваются пакетами только в определённые временные слоты. Типичные значения: 7.5–4000 мс. Чем больше интервал, тем ниже средняя скорость, но выше энергоэффективность.
4. **MTU (Maximum Transmission Unit):** определяет максимальный размер пакета протокола ATT. Для **Classic Bluetooth** через RFCOMM типичный размер кадра составляет от 255 до 1011 байт (при базовом MTU L2CAP в 672 байта), вплоть до 65535 байт. Для **BLE** стандартный MTU по умолчанию составляет 23 байта (из которых 3 байта — заголовок ATT, оставляя 20 байт для данных). В современных реализациях (Bluetooth 4.2+) через процедуру *MTU Exchange* это значение обычно увеличивают до 247 байт для оптимального соответствия максимальному размеру PDU канального уровня (251 байт), однако ОС Android поддерживает MTU до 517 байт.

### 6.3.3 Особенности измерений пропускной способности

Для получения воспроизводимых результатов необходимо:

- Использовать фиксированный размер полезной нагрузки.
- Проводить измерения в течение достаточного интервала (не менее 10 секунд) для усреднения флуктуаций.
- Фиксировать параметры соединения: интервал, MTU, режим подтверждения.

### 6.3.4 Структура пакетов Bluetooth

Для понимания механизмов передачи данных необходимо рассмотреть структуру пакетов на канальном уровне для каждого из режимов работы Bluetooth.

**Пакеты Basic Rate (BR).** Общий формат пакетов BR Bluetooth показан на рис. 1. Каждый пакет состоит из трёх основных полей:



- **Access Code** (68 или 72 бита) — код доступа для синхронизации и идентификации пикосети;
- **Header** (54 бита) — заголовок пакета, содержащий информацию об адресате, типе пакета и контроле ошибок;
- **BR Payload** (0–2790 бит) — полезная нагрузка, содержащая передаваемые данные.

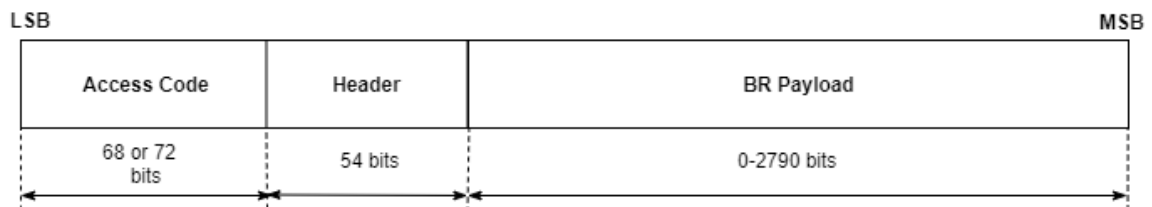


Рис. 1: Структура пакета Bluetooth Basic Rate (BR)

Все поля пакета BR модулируются с использованием GFSK.

**Пакеты Enhanced Data Rate (EDR).** Общий формат пакетов EDR Bluetooth показан на рис. 2. Структура включает:

- **Access Code** и **Header** — аналогичны пакетам BR и модулируются GFSK для обеспечения совместимости;
- **Guard Time** (4.75–5.25 мкс) — защитный интервал для переключения между схемами модуляции;
- **Sync Sequence** (11 мкс) — синхронизирующая последовательность для настройки приёмника на новую схему модуляции;
- **EDR Payload** (0–2790 бит) — полезная нагрузка, модулируемая методами  $\pi/4$ -DQPSK или 8DPSK;
- **Trailer** (2 символа) — завершающее поле для корректного завершения передачи.

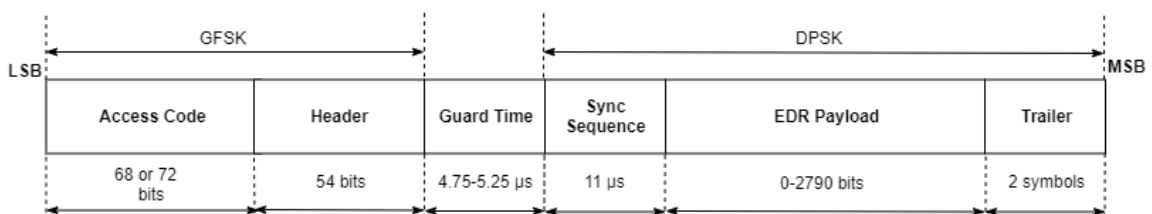


Рис. 2: Структура пакета Bluetooth Enhanced Data Rate (EDR)

Такое разделение позволяет использовать надёжную модуляцию GFSK для служебных полей и высокоскоростную фазовую модуляцию для данных.

**Пакеты BLE незакодированный PHY (LE 1M и LE 2M).** Спецификация ядра Bluetooth задаёт два физических уровня для незакодированного PHY: LE 1M (1 Мсимв/с) и LE 2M (2 Мсимв/с). Структура пакета показана на рис. 3:

- **Preamble** (1 или 2 октета) — преамбула для синхронизации по частоте;
- **Access Address** (4 октета) — адрес доступа, идентифицирующий соединение или рекламный канал;
- **Protocol Data Unit (PDU)** (2–258 октетов) — блок данных протокола, содержащий заголовок и полезную нагрузку;
- **Cyclic Redundancy Check (CRC)** (3 октета) — контрольная сумма для обнаружения ошибок;
- **Constant Tone Extension** (16–160 мкс, опционально) — расширение с постоянной тональностью для определения направления сигнала (AoA/AoD).

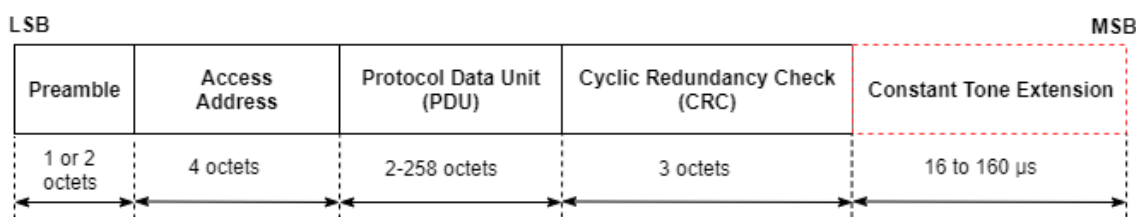


Рис. 3: Структура пакета BLE незакодированный PHY (LE 1M/LE 2M)

**Пакеты BLE закодированный PHY (Coded PHY).** Закодированный PHY реализуется для пакетов BLE на всех физических каналах и использует избыточное кодирование FEC (Forward Error Correction) для увеличения дальности связи. Структура пакета показана на рис. 4:

- **Preamble** (80 мкс) — преамбула;
- **Access Address** (256 мкс) — адрес доступа;
- **Coding Indicator (CI)** (16 мкс) — индикатор кодирования, указывающий коэффициент расширения ( $S=2$  или  $S=8$ );
- **TERM1** (24 мкс) — первый терминальный блок FEC;
- **PDU** ( $N \times 8 \times S$  мкс) — блок данных протокола, где  $S$  — коэффициент кодирования;
- **CRC** ( $24 \times S$  мкс) — контрольная сумма с кодированием;
- **TERM2** ( $3 \times S$  мкс) — второй терминальный блок FEC.

Пакет разделён на два FEC-блока: FEC Block 1 включает CI и TERM1,

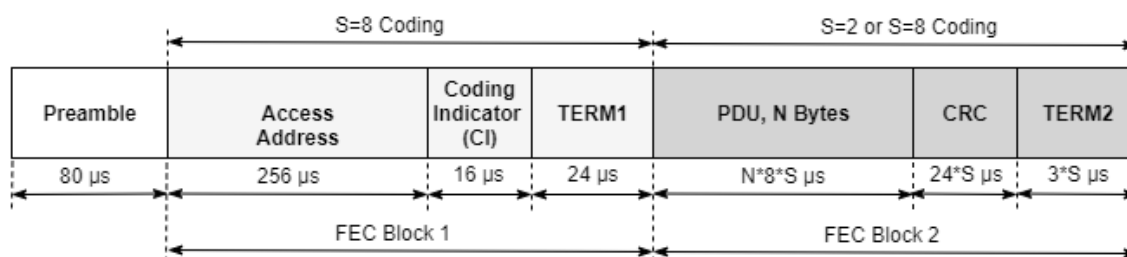


Рис. 4: Структура пакета BLE закодированный PHY (Coded PHY)

FEC Block 2 включает PDU, CRC и TERM2. При  $S=8$  каждый бит данных повторяется 8 раз, что значительно повышает помехоустойчивость ценой снижения скорости.

### 6.3.5 Влияние протоколов верхних уровней на пропускную способность

## 6.4 Порядок выполнения лабораторной работы

**Оборудование:** два модуля ESP32, ПК с установленной Arduino IDE и драйверами, два окна терминала для мониторинга последовательного порта.

**Программное обеспечение:**

- Для Classic Bluetooth: прошивки ESP32\_BT\_TX.ino (передатчик) и ESP32\_BT\_RX.ino (приёмник).
- Для Bluetooth Low Energy: прошивки ESP32\_BLE\_TX.ino (передатчик) и ESP32\_BLE\_RX.ino (приёмник).

### 1. Измерение скорости для Classic Bluetooth (BR/EDR) при различных размерах пакета:

- a) Откройте скетч ESP32\_BT\_TX.ino и найдите параметр размера отправляемого блока:

```
const size_t CHUNK = 2048;
```

- b) Последовательно установите значения CHUNK: 512, 1024 и 2048 байт. Для каждого значения выполните следующие действия:

- Скомпилируйте и загрузите прошивку в модуль-передатчик (приёмник ESP32\_BT\_RX.ino перепрошивать не требуется).
- Откройте два окна Serial Monitor (115200 бод) для TX и RX.
- Перезагрузите оба устройства. Дождитесь сообщения [TX] Connected! Starting test (10 sec)....
- По окончании теста зафиксируйте вывод передатчика вида:

```
[TX] 2048000 bytes | 10000 ms | 1638.4 kbps | 204.8 KB/s
```

- v) Повторите измерение 3 раза для каждого размера пакета, вычислите среднее значение скорости.

## 2. Измерение скорости для Bluetooth Low Energy при различных значениях MTU:

- a) Откройте скетч ESP32\_BLE\_TX.ino и найдите строку установки максимального размера атрибута:

```
pClient->setMTU(517);
```

- b) Последовательно установите значения MTU: 23 (стандартное), 128, 247 и 517 байт. Для каждого значения выполните:

- i) Скомпилируйте и загрузите прошивку в модуль-передатчик (приёмник ESP32\_BLE\_RX.ino остаётся без изменений).
- ii) Откройте Serial Monitor для обоих устройств.
- iii) Перезагрузите устройства. Дождитесь завершения 10-секундного теста.
- iv) Зафиксируйте вывод приёмника вида:

```
[RX] Test complete. Received: 122880 bytes. Duration:
10.02 sec. Speed: 12.23 KB/s
```

- v) Повторите измерение 3 раза для каждого MTU, вычислите среднее значение.

## 3. Сравнительный анализ:

- Заполните таблицы результатов.
- Постройте графики зависимости скорости передачи данных от размера полезной нагрузки (CHUNK) для BR/EDR и от согласованного MTU для BLE.
- Проанализируйте полученные данные: как изменение размера пакета влияет на эффективность канала? С какими ограничениями протоколов RFCOMM и ATT связаны наблюдаемые пределы роста скорости? Оцените долю служебных заголовков в общем трафике.

## 6.5 Содержание отчета

1. Заголовок согласно приложению.

Таблица 6.1 — Результаты измерений пропускной способности Classic Bluetooth (BR/EDR)

| <b>CHUNK (байт)</b> | <b>Мин. скорость<br/>(кбит/с)</b> | <b>Макс. скорость<br/>(кбит/с)</b> | <b>Средняя скорость<br/>(кбит/с)</b> | <b>Отклонение (%)</b> |
|---------------------|-----------------------------------|------------------------------------|--------------------------------------|-----------------------|
| 512                 |                                   |                                    |                                      |                       |
| 1024                |                                   |                                    |                                      |                       |
| 2048                |                                   |                                    |                                      |                       |

Таблица 6.2 — Результаты измерений пропускной способности Bluetooth Low Energy

| <b>MTU (байт)</b> | <b>Мин. скорость<br/>(кбит/с)</b> | <b>Макс. скорость<br/>(кбит/с)</b> | <b>Средняя скорость<br/>(кбит/с)</b> | <b>Отклонение (%)</b> |
|-------------------|-----------------------------------|------------------------------------|--------------------------------------|-----------------------|
| 23 (default)      |                                   |                                    |                                      |                       |
| 128               |                                   |                                    |                                      |                       |
| 247               |                                   |                                    |                                      |                       |
| 517               |                                   |                                    |                                      |                       |

\* — максимальное значение MTU, поддерживаемое данной реализацией ESP32

2. Цель работы.
3. Краткое описание использованных прошивок (ссылки на файлы ESP32\_BT\_TX.ino, ESP32\_BT\_RX.ino, ESP32\_BLE\_TX.ino, ESP32\_BLE\_RX.ino).
4. Таблица результатов измерений.
5. Графики сравнения пропускной способности.
6. Анализ влияния параметров (размер пакета, режим подтверждения) на скорость.
7. Ответы на контрольные вопросы.

**Примечание.** При оформлении отчёта допускается приводить только ключевые фрагменты вывода Serial Monitor. Полные логи можно приложить отдельным файлом.

### **Контрольные вопросы**

1. Почему реальная пропускная способность канала всегда ниже сырой скорости физического уровня?
2. Какие накладные расходы вносит протокол RFCOMM при передаче данных в классическом Bluetooth?
3. Почему даже при одинаковой сырой скорости 1 Мбит/с пропускная способность BLE значительно ниже, чем у BR?
4. Как размер пакета (payload) влияет на эффективность передачи в протоколе ATT (BLE)?
5. В чём преимущество режима Write Without Response в BLE и когда его целесообразно использовать?
6. Как интервал соединения (Connection Interval) влияет на среднюю скорость и энергопотребление в BLE?
7. Почему в сценариях передачи больших объёмов данных (файлы, изображения) предпочтительнее использовать EDR, а не BLE?
8. Какие факторы окружающей среды могут существенно снизить скорость передачи в беспроводных каналах?